

Algorithm Use-Case

The baseline algorithm is an implementation of a “peak-normalized asymmetric filter”. It is designed as a pipeline for sensor data conditioning and feature extraction. The provided algorithm applies a linear pre-scaling to normalize the range of the input, uses an asymmetric 4-tap FIR filter to emphasize specific features and suppress noise, and employs a Leaky ReLU activation function to provide non-linear shaping. This serves to make the output signal better suited for neural network classifiers. The overall function is to take some input signal data, normalize it, detect peaks for gain adjustment, and shape the signal for use in classifiers.

How is FPGA Acceleration Beneficial?

FPGA acceleration is beneficial for this algorithm because there is a large potential for parallelism and deterministic latency requirement. We can use the FPGA to create pipelined stages for processing signal samples sequentially in real time. Additionally, the use of an FPGA helps us tackle the fork-join dependency created by the block-based peak detection by using our FIFOs and on-chip BRAM.

Calculated Latency

1194536 cycles * 6.908 ns = 8.25 ms latency

Report Screenshots

```
Report time      : Mon Mar 30 13:50:28 EDT 2026.
Solution         : hls.
Simulation tool  : xsim.
```

		Latency(Clock Cycles)			Interval(Clock Cycles)			Total Execution Time
RTL	Status	min	avg	max	min	avg	max	(Clock Cycles)
VHDL	NA	NA	NA	NA	NA	NA	NA	NA
Verilog	Pass	1194536	1194536	1194536	NA	NA	NA	1194536

```
Implementation tool: Xilinx Vivado v.2025.1.1
Project:            project_1
Solution:           hls
Device target:      xczu3eg-sbva484-1-e
Report date:        Mon Mar 30 14:01:20 EDT 2026
```

=== Post-Implementation Resource usage ===

```
SLICE:              0
LUT:                3643
FF:                3023
DSP:                8
BRAM:              362
URAM:              0
LATCH:             0
SRL:               141
CLB:               886
```

=== Final timing ===

```
CP required:        10.000
CP achieved post-synthesis: 5.538
CP achieved post-implementation: 6.908
Timing met
```